Serial Game Server

Plan of action for our project meeting · EEE3095S CS Project 2026

DUE	TIME LEFT	TEAM	LATE PENALTY	BONUS
27 Oct, 23:59	21 days from 6 Oct	2 students, A and B	10% per day	up to 5, total capped at 100

The plan in one paragraph

We build a **Tic-Tac-Toe server on the STM32** that owns the board, checks every request and replies over UART, plus a thin **PC GUI** that only draws what the server confirms. We freeze the protocol first, build the game logic as portable C that we can test on a PC, test everything against a **mock server** before the board is involved, then integrate on hardware, then add the stretch features (server-side AI, hardware-timer move clock, undo). We work in **stages and take turns** (one drives with Claude Code, the other reviews), and every stage ends by writing its state into the repo so either of us can continue from the files, not from memory.

Why this plan: the marks follow the **report headings**, and most of them reward correctness, robustness and **evidence**, not a complicated game. A simple game finished early leaves time for tests, diagrams and a good report.

What the marks reward, and where each mark comes from

Report section	Marks	What earns it	Stage
Introduction	10	Design choices, task allocation, report structure	0, 10
Requirements and message protocol	15	Framing, encoding, every command and error, departures from the brief, sequence diagram	1
Specification and design	15	Server FSM, GUI flowchart, circuit diagram	1, 4, 8
Implementation	15	Explained code excerpts, especially the non-obvious parts	5, 6
Game logic and server authority	15	Validation, turn control, win/draw, proof a rejected request cannot change state	2
Protocol robustness and integrity	10	Framing, specific NACKs, recovery from partial or garbage input	5, 7
Validation and test harness	15	Passing automated tests, plus screenshots or a short video	3, 7, 10
Conclusion	5	Success, usefulness, limitations, improvements	10
Bonus (stretch features)	up to 5	AI opponent, timer move clock, undo	9

Code and a demo are also submitted, and the report must link to the GitHub repo and a demo video. Report limit: 10 pages in IEEE conference format, references included.

1. What we are building

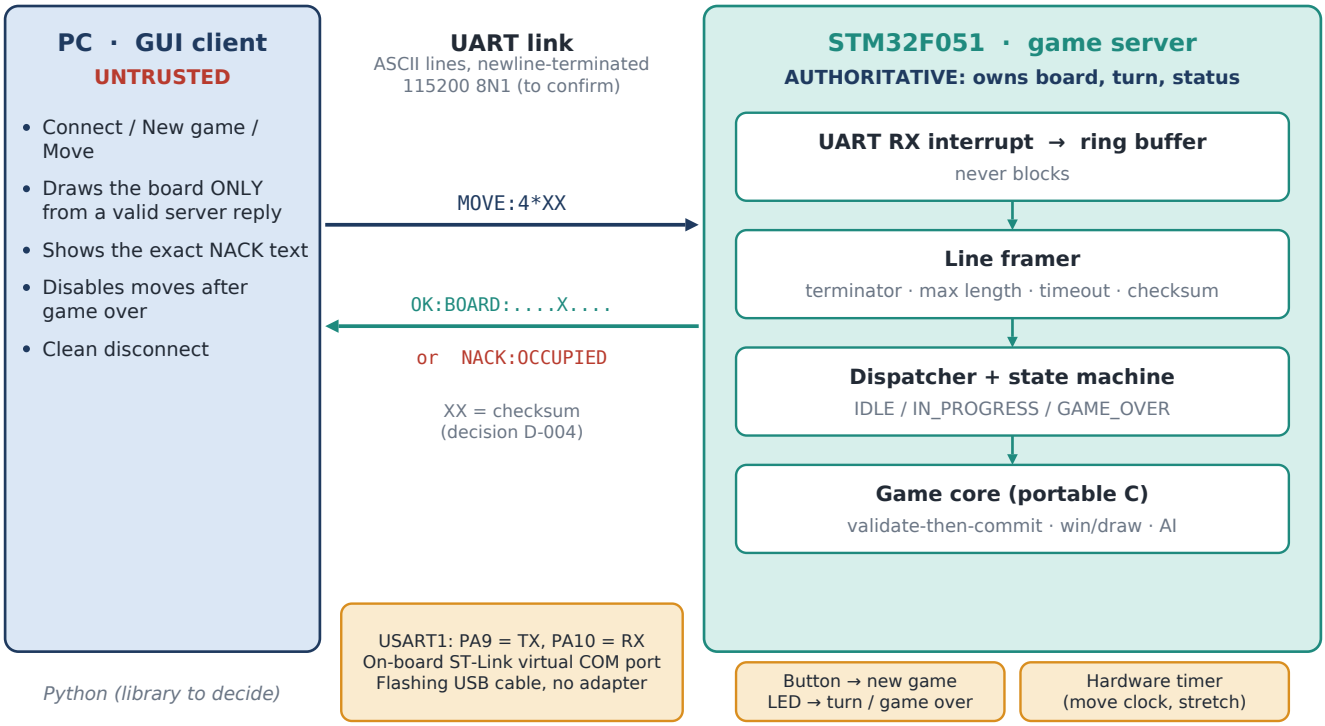


Figure 1. System architecture. The STM32 is the single source of truth; the PC never decides whether a move is legal.

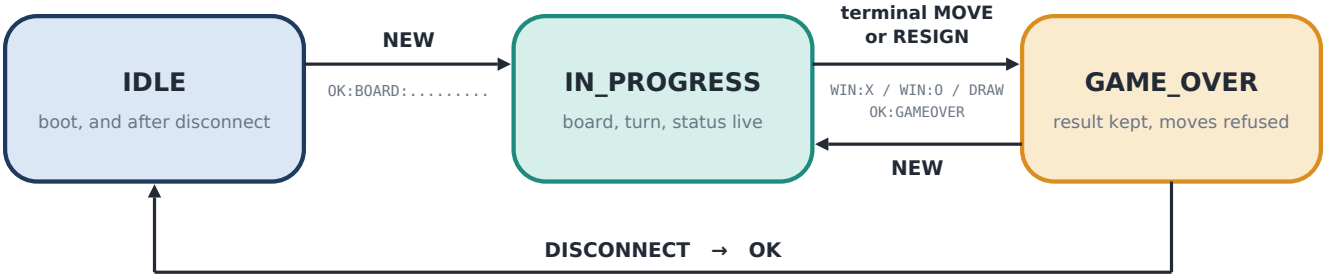
Responsibility	Lives on	Why
Board, turn, game status	**STM32**	Single source of truth; the client cannot corrupt it.
Move legality, win/draw, AI move	**STM32**	Server authority is a marked requirement; the AI must never be illegal.
Framing, checksum, timeout, NACKs	**STM32**	Garbage or partial input must not crash or block the server.
Drawing the board, buttons, messages	**PC**	Redraw only from a valid server reply, never optimistically.
Automated tests	**PC** (pytest)	Drive the real serial interface; the same tests run against the mock server and the board.

2. How the server behaves

IDLE accepts
CONNECT → OK
NEW → game starts
else → NACK:NOGAME

IN_PROGRESS accepts
MOVE ok → OK:BOARD
MOVE bad → NACK (no change)
STATE · RESIGN

GAME_OVER accepts
NEW · STATE · DISCONNECT
MOVE → NACK:NOGAME



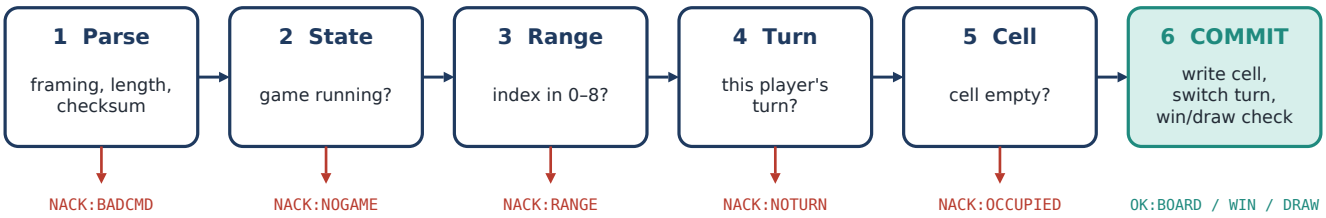
Hardware: the on-board button triggers a local NEW game (reset). The LED shows whose turn it is and blinks after game over.

Malformed, oversized or garbage input → NACK:BADCMD from every state; state is left unchanged.

Figure 2. Server state machine (draft, frozen at Stage 1). This diagram is compulsory in the report.

Validate first, change state last

Nothing is written to the board until every check has passed (checks 1–5 are read-only).



Order of checks 2–5 is frozen in protocol.md at Stage 1.

Figure 3. Every MOVE passes through the same read-only checks before anything is written. This is the basis of the report's proof that rejected requests cannot change state.

3. The stages

Each stage has one driver (prompts Claude Code) and one navigator (reviews, signs off). A stage is done only when the tests have been run, both partners can explain the code, and the docs are updated. Stages 2 and 3 can run in parallel because neither needs the board (stages 4 to 7 do).

#	Stage	Driver	Done when…	Produces
0	**Setup**	Both	Repo on GitHub, partners named, decisions agreed, project plan written.	Plan (A.1), allocation
1	**Protocol and FSM spec**	A	protocol.md complete: framing, max length, timeout, checksum, every command/error, state table. Both partners review and freeze.	A.2, FSM diagram, sequence diagram
2	**Portable game core**	B	game.c/.h with no HAL calls; PC unit tests cover all 8 win lines, draw, every rejection; a rejected move leaves the board byte-identical.	Game-logic section draft
3	**Mock server and pytest**	A	Python mock speaks the protocol; pytest covers every required case plus garbage input; `--port` option.	Test list, first evidence
4	**Hardware bring-up**	B	Transmit test and line-echo test work over the ST-Link virtual COM port (USART1, PA9/PA10); AF number and port name recorded; button and LED confirmed.	Circuit diagram (A.3), bring-up log
5	**Firmware framing**	A	RX interrupt, ring buffer, framer with oversize and timeout handling; no blocking; nothing extra on the game UART.	Robustness section draft
6	**FSM and dispatcher**	B	Command handlers on game core; NOGAME/NOTURN enforced; button starts a new game; LED shows turn and game over.	Code excerpts
7	**Hardware integration**	A	Full pytest suite passes on the real board; garbage and partial-frame recovery captured.	Test output, serial captures
8	**GUI**	B	Connect, new game, move, specific NACK text, win/draw notice, moves disabled after game over, clean disconnect.	GUI flowchart, screenshots
9	**Stretch features**	A then B	AI (test: never illegal), then hardware-timer move clock, then undo. Each has tests; protocol.md updated.	Bonus evidence
10	**Freeze and package**	Both	Code freeze, demo video, IEEE report within 10 pages, archive tested in a clean folder.	Final report, video link

4. Timeline

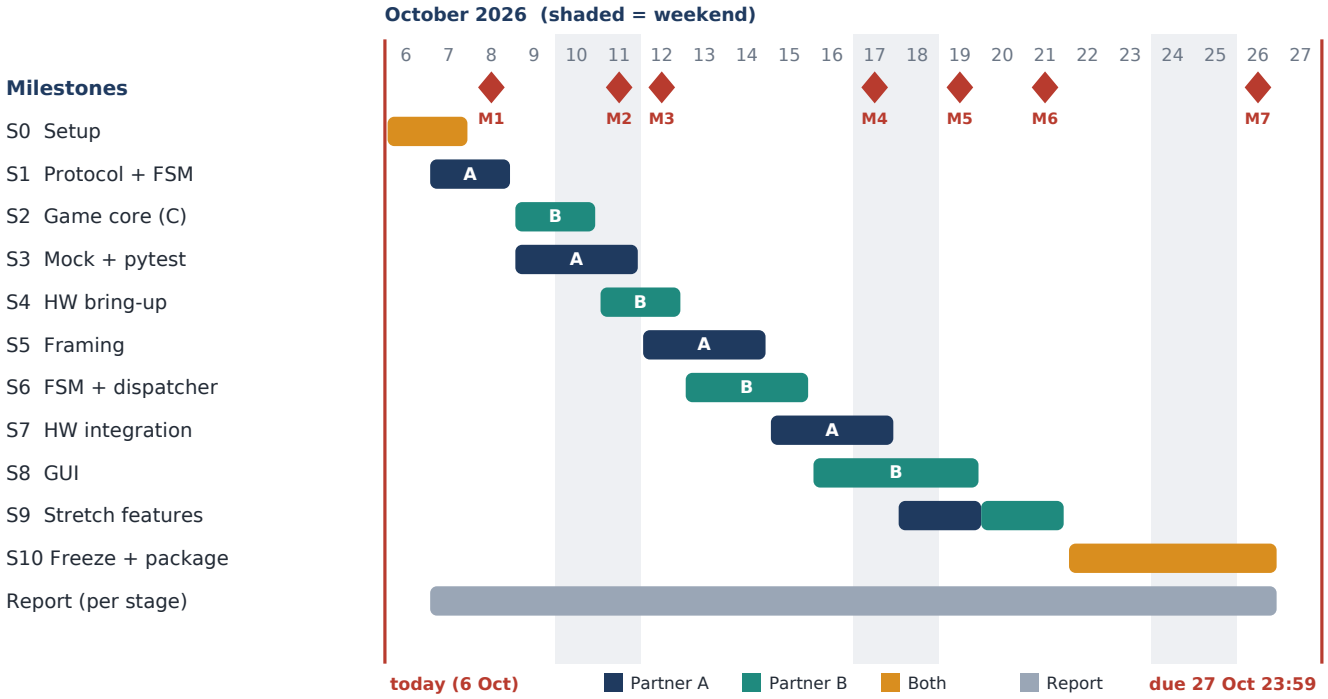


Figure 4. Plan from today to the deadline. Bars show who drives; overlapping bars are different people working in parallel.

Milestone	By end of	Check
M1 Protocol frozen	8 Oct	Both partners have read and agreed protocol.md and the FSM.
M2 Required tests pass on the mock	11 Oct	Every bullet of the brief's test list is a passing pytest.
M3 UART link works	12 Oct	Board echoes a line from the PC over the ST-Link serial port; AF number and port name recorded.
M4 Full suite passes on the board	17 Oct	Same pytest, real serial port, output saved to evidence.
M5 GUI complete	19 Oct	All required GUI behaviours work against the real board.
M6 Stretch done, code freeze	21 Oct	After this only the report, video and packaging change.
M7 Report and video finished	26 Oct	Archive tested in a clean folder; 27 Oct is buffer only.

The deadline is Tuesday 27 Oct and the late penalty is 10% per day, so we treat 26 Oct as the real deadline. The report is drafted a section at a time during each stage rather than at the end.

5. How we take turns with Claude Code

We use one chat, but the **repo is the memory**. Chats get long and either of us may start a new one, so every turn ends by writing its state to files. Any turn then starts with: *“Read docs/HANDOFF.md and CLAUDE.md, then continue with the next stage.”*

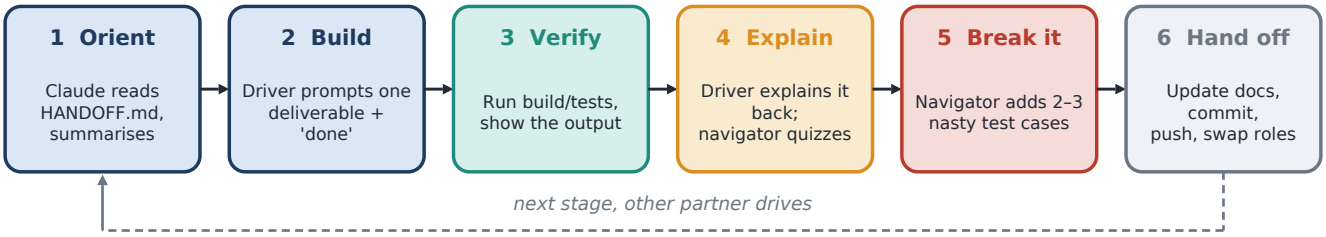


Figure 5. The six steps of every stage. The driver prompts; the navigator reviews and signs off.

File	What it holds	Rule
`CLAUDE.md`	Project rules: server authority, validate-then-commit, firmware limits, commands.	Short; update when a rule changes.
`docs/HANDOFF.md`	Done, next, open problems, build/test commands.	Overwritten at the end of every turn.
`docs/decisions.md`	Every choice and why (feeds report section A.4).	Append-only.
`docs/protocol.md`	Framing, commands, errors, state table.	Single source of truth; changed only by agreement, in the same commit as code.
`docs/evidence/`	Test output, serial captures, screenshots, wiring photos.	Saved as we go, named by stage.

Ground rules

- **Understand every line.** The driver explains each module back in their own words and the navigator can ask Claude to quiz them. The demo and the code-excerpt marks depend on this. If we cannot explain it, we ask for a simpler version.
- **Never trust “tests pass”.** We run them and read the output. The navigator checks that tests assert what the brief requires.
- **Navigator breaks it.** Two or three nasty cases per stage (garbage, wrong state, off-by-one), each becomes a test.
- **One writer per file.** Parallel work is split by folder (firmware, gui, tests) and by branch. Pull before starting, push before stopping.
- **No protocol changes mid-stage.** Raise it, update protocol.md, agree, then change code.
- **Only one board.** Whoever holds it pastes real serial output back to Claude when something fails; Claude cannot see the hardware.
- **Disclose.** The report references list the AI assistance and the reused course Practical 2A base.

6. Decisions to make in the meeting

#	Decision	Recommendation	Why	OK?
D-001	**Game**	Tic-Tac-Toe	Marks reward correctness and evidence, not complexity. Connect-4 only if this is solid early.	☐
D-002	**Stretch order**	AI, then timer, then undo	AI is cheap on 3×3 and testable; bonus is capped at 5, so it must not endanger the core.	☐
D-003	**UART access**	CMSIS registers (no HAL UART)	The course base has no HAL UART driver; registers suit an interrupt and ring buffer and are easy to explain.	☐
D-004	**Checksum**	XOR, written *XX	Simpler to explain and test than CRC-8; both meet the brief's recommendation.	☐
D-008	**UART path**	ST-Link virtual COM, USART1 PA9/PA10	Settled: the lecturer confirmed the newer board needs no adapter. Same USB cable as flashing. Nothing to vote on.	✔
New	**GUI library**	Tkinter	Ships with Python, nothing to install, easy for markers to run. PyQt5 looks nicer but adds an install step.	☐
New	**Roles A / B and repo owner**	Decide names now	A drives stages 1, 3, 5, 7; B drives 2, 4, 6, 8. Repo link must be viewable by markers.	☐

UART path: settled

Our lecturer confirmed that on the newer UCT dev board **USART1 (PA9 = TX, PA10 = RX) is already wired to the on-board ST-Link**, which appears as a serial port on the PC over the same USB cable we flash with. We do **not** use the STM32USB2UART adapter (it should be returned). This matches Board.md. We still confirm three things on the bench in Stage 4: the alternate-function number from the datasheet table, the PC port name (it changes per machine and socket, so tests take `--port`), and that only one program holds the port at a time.

Main risks

Risk	What we do about it
Only one board between two people	Game core and GUI are developed against a PC mock; the board is needed only from stage 4 to stage 7.
Serial port name changes, or a terminal still holds the port	Tests take --port; close PuTTY/screen before pytest; log each attempt in bringup-log.md.
Report left to the end (most marks)	Draft one section per stage; keep evidence as we go; 26 Oct is the real deadline.
Protocol drifts between us	protocol.md is frozen at M1 and changed only by agreement.
Code we cannot explain	Explain-back and quiz each stage; partners swap walkthroughs after every merge.

Suggested agenda (30 min)	Time
1. Walk through the goal, the marks map and the architecture (Figures 1–3)	5
2. Agree decisions D-001 to D-004, GUI library and roles	10
3. Check the timeline and milestones against our other deadlines	5
4. Assign the actions below and book the next check-in	10

Actions before the next meeting	Owner	Done
Create GitHub repo, unzip the Stage 0 skeleton, push		☐
Fill in partner names in HANDOFF.md; write project plan (A.1)		☐
Flash the placeholder firmware (blinks the green LED) to confirm the toolchain and board work		☐
Open the ST-Link serial port in a terminal and note the port name for each of your computers		☐
Start Stage 1: protocol and FSM spec	A	☐